

Full Stack Open Part 0: The Verdict

The purpose of Part 0 is to teach the **evolution of web development** so that you understand exactly *what problem React solves* before you write a single line of React code.

Here is what the three exercises are meant to drill into your brain:

1. Exercise 0.4 (The Traditional Web)

- **The Lesson:** This is how the web used to work (and how older frameworks like PHP or Ruby on Rails still often work).
- **The Takeaway:** The server is the boss. The browser is "dumb." Every time you do an action (like submit a form), the server forces the browser to throw away the entire page and download *everything* again (HTML, CSS, JS, JSON).
- **The Problem:** It's slow, inefficient, and creates a "flickering" user experience because the page has to constantly reload.

2. Exercise 0.5 (The SPA Initial Load)

- **The Lesson:** This is the foundation of modern web apps (React, Vue, Angular).
- **The Takeaway:** Instead of the server sending a fully built HTML page, the server sends a mostly empty HTML file and a **massive JavaScript file** (`spa.js`).
- **The Shift in Power:** The browser is now "smart." The JavaScript file takes over. From this point forward, JavaScript handles all the logic, fetches the raw data (JSON), and builds the UI directly in the browser.

3. Exercise 0.6 (The SPA Action)

- **The Lesson:** This is the magic of Single Page Applications (SPAs).
- **The Takeaway:** When you submit a form, the page **does not reload**. JavaScript intercepts the action (e.g. `preventDefault()`), updates the screen instantly using the data it already has in memory, and quietly sends a single `POST` request to the server in the background to save it.
- **The Solution:** It's incredibly fast. The user gets instant feedback, and we only send a tiny piece of JSON data over the network instead of downloading the whole website again.

The Final Verdict: Why did they make you do this?

If you jump straight into React without understanding this, React just feels like a confusing way to write HTML in JavaScript.

But because you did these diagrams, you now understand that **React is just a tool designed to do exactly what you saw in 0.5 and 0.6.**

React exists to take control of the browser, stop page reloads, manage data (state) in memory, and update the screen instantly without needing the server to send new HTML.

You now understand the *why*. You are officially ready to learn the *how*.

React Fundamentals

Full Stack Open - Part 1a Notes

1. Traditional vs. Modern Web Apps

Traditional Web Pages: Behave like a constant loop of POST/GET requests. Every action forces the server to reload the entire page, sending HTML, CSS, and JS over and over. This is slow and inefficient.

Modern Single Page Apps (SPA): Solves this by fetching the HTML/CSS/JS exactly *once*. After that, React acts as the engine, rendering components dynamically on the client-side and only talking to the server to send/receive raw JSON data.

2. Initializing React with Vite

To write React, you need background tools (local server, compiler, etc.). Setting this up manually is a nightmare. **Vite (pronounced "veet")** handles all this setup automatically.

The Setup Command:

```
npm create vite@latest
```

- **Project name:** `part1`
- **Framework:** `React`
- **Variant:** `JavaScript`
- **Linters:** `ESLint` (The industry standard for catching errors early)

Starting the Server:

Vite might ask *"Install with npm and start now?"* If you say no, you must run these manually:

```
cd "folder-name"
npm install      // Downloads React and Vite into node_modules
npm run dev      // Starts the local development server
```

Cleanup Rule: For plain React apps, delete `App.css`, `index.css`, and the `assets` folder generated by Vite.

3. Components & Architecture

What is a Component? It is simply a JavaScript function that returns JSX (HTML). Think of it as a custom HTML tag you invent (e.g., `<Header />`). It makes code reusable and organized like Lego blocks.

The Core Files:

- **App.jsx:** This is your main component. All content to be rendered goes here or inside components imported here.
- **main.jsx:** The "middle-man". You do not edit this. It imports **App.jsx** and injects it into the actual **index.html** file using `createRoot(...).render(...)`.

4. JSX (JavaScript XML)

JSX is HTML written directly inside a JavaScript file. It lets you inject dynamic JS variables using curly braces `{}`.

The Secret of JSX: Browsers do not understand JSX. If sent directly to Chrome, it would crash. Behind the scenes, the compiler turns your beautiful JSX into ugly JavaScript.

What you write (JSX):

```
const App = () => {  
  return (  
    <div>  
      <p>Hello</p>  
    </div>  
  )  
}
```

What the browser actually sees:

```
const App = () => {  
  return React.createElement(  
    'div',  
    null,  
    React.createElement('p', null,  
      'Hello')  
  )  
}
```

Why use it? It completely removes the need for painful DOM manipulation (like `document.createElement`) while keeping the massive power of JavaScript.

5. Props (Passing Data)

Props allow you to pass dynamic data from a parent component down to a child component.

- **The Unwritten Rule:** Always name the parameter **props**. Never name it **data** or **heading**.
- **Naming Convention:** Always use **camelCase** (e.g., `userName`, `userAge`).

```
// 1. Defining the Component to receive props  
const Hello = (props) => {  
  return (  
    <div>  
      <p>Hello {props.name}, you are {props.age} years old!</p>  
    </div>  
  )  
}
```

```
    </div>
  )
};

// 2. Passing the data from the Parent
const App = () => {
  let userName = "Vinayak";
  return (
    <div>
      <Hello name={userName} age={23} />    <!-- Passing variable and number -->
      <Hello name="Varoon" age={25} />      <!-- Passing hardcoded string -->
    </div>
  )
};
```

Prop Drilling: The architecture of passing props from `App` → `Content` → `Part`. This is how complex data moves through a React application.

6. Golden Rules & Common Errors

The 3 Golden Rules of Components

1. **Capitalize Component Names:** If you name it `hello`, React thinks it's a regular HTML tag. It must be `Hello`.
2. **One Root Element:** A component can only return ONE parent wrapper. You cannot return two elements side-by-side. Wrap them in a `<div>` or an empty Fragment `<> ... </>`.
3. **Never Nest Definitions:** Never define a component inside another component's curly braces. Keep `const App` and `const Hello` completely separate.

The "Object rendering" Error

Error: Objects are not valid as a React child (found: object with keys {name, age}).
If you meant to render a collection of children, use an array instead.

Why it happens: You tried to print an entire JavaScript Object inside curly braces `{}` in your JSX. React doesn't know how to draw an object.

The Fix: Never render an object directly. Always drill down to the specific primitive value (string or number) you want to show, like `{props.name}`.

The ESLint Prop-Types Warning

ESLint throws a warning if you accept props without explicitly defining their data types. Since type-checking isn't needed until later in the course, silence it by adding this to the `rules:` section of

eslint.config.js:

```
"react/prop-types": 0
```

Generated for Full Stack Open - Strict Mentorship Session

Component State & Event Handlers (Part 1c)

1. Destructuring Props & Helper Functions

Helper Functions: Defining functions (like `bornYear`) inside a component is a standard React practice. It keeps the logic scoped precisely to where it is needed.

Destructuring Props: Instead of writing `props.name` everywhere, we extract the values directly in the function parameters. This acts as instant documentation for what the component needs.

```
// ✅ Professional Standard: Destructuring in parameters
const Hello = ({ age, name }) => {

  // Implicit Return (One-liner helper function)
  const bornYear = () => new Date().getFullYear() - age;

  return (
    <div>
      <p>Hello {name}, you are {age} years old</p>
      <p>so you were probably born in {bornYear()}</p>
    </div>
  )
};
```

2. Stateful Components (useState)

Why normal variables fail: If you use `let counter = 0;`, React does not care if the variable changes. The screen will never redraw.

The React Flare Gun: React needs you to shoot a flare into the sky to say *"My data changed! Redraw the screen!"* That flare gun is `useState`.

```
import { useState } from 'react'

const [counter, setCounter] = useState(0);
```

- **Convention:** The first variable is the data (`counter`). The second variable is the exact same word, prefixed with `set` (`setCounter`).
- **Fixed Order:** Index 0 is ALWAYS the value. Index 1 is ALWAYS the remote control function.
- **Use Case:** If data never changes on the screen, don't use state. If data DOES change (like a like button or text input) and the user must see it instantly, use `useState`.

3. Event Handling (The Infinite Loop Trap)

An event handler must be a **function reference**, not a function call. Never execute the handler directly inside the JSX.

✗ BAD (Infinite Loop Crash)

```
<button onClick={increaseByOne()}>
```

The `()` causes the function to run instantly on page load. It updates state, triggers a re-render, calls the function again... resulting in an endless loop.

✓ GOOD (Pass Reference)

```
<button onClick={increaseByOne}>
```

Pass the name of the function. It waits patiently until the user actually clicks.

4. React Architecture

Lifting State Up: Data in React is a waterfall—it only flows DOWN. If sibling components (like `<Display />` and `<Button />`) need to share the same data, you cannot put `useState` inside either of them. You must move the state **UP** to their closest common parent (like `App`), and pass the data down via props.

Component Reusability: Create clean, tiny components like `<Button />` and reuse them by passing different props (e.g., passing different `text` and `onClick` handlers to the exact same `Button` blueprint).

5. The Most Important Rule: Changes in state cause re-rendering

Every single time you use a `set...` function, React re-runs your entire component function from top to bottom.

```
import { useState } from "react";

const Display = ({ counter }) => {
  return <div>{counter}</div>
};

const Button = ({ onClick, text }) => {
  return <button onClick={onClick}>{text}</button>
}

const App = () => {
  const [counter, setCounter] = useState(0)
  console.log('rendering with counter value', counter)

  const increaseByOne = () => {
```

```
    console.log('increasing, value before', counter)
    setCounter(counter + 1)
  }

  const decreaseByOne = () => {
    console.log('decreasing, value before', counter)
    setCounter(counter - 1)
  }

  const setToZero = () => {
    console.log('resetting to zero, value before', counter)
    setCounter(0)
  }

  return (
    <div>
      <Display counter={counter} />
      <Button onClick={increaseByOne} text="plus" />
      <Button onClick={setToZero} text="zero" />
      <Button onClick={decreaseByOne} text="minus" />
    </div>
  )
};

export default App;
```

1. **Initial Render:** React calls `App()` for the first time. `counter` is 0. It paints the HTML. React goes to sleep.
2. **State Change:** The user clicks "plus". The handler calls `setCounter(0 + 1)`. The flare gun is fired.
3. **Re-render Magic:** React wakes up, realizes the HTML is outdated, and re-runs the entire `App()` function. It skips the initial 0, sees the counter is 1, hits the console logs, generates brand new HTML, and paints it on the screen instantly.

6. Debugging & StrictMode

Why is the console printing twice? React's `<StrictMode>` intentionally renders components twice in development mode. It acts as a safety inspector, stress-testing your code to ensure there are no bugs or unpredictable side-effects.

Seeing two console logs is 100% normal and proves your app is working correctly. It is automatically removed when you deploy your app to production.

A more complex state, debugging React apps

Complex State

- In our previous example, the application state was simple as it was comprised of a single integer. What if our application requires a more complex state?
- Default and basic declaration of multiple state;

```
const [left, setLeft] = useState(0);  
const [right, setRight] = useState(0);
```

- Improvised declaration

```
const [clicks, setClicks] = useState({ left: 0, right: 0 })
```

- stateHandlers

```
const handleLeftClick = () => {  
  const newClicks = {  
    left: clicks.left + 1,  
    right: clicks.right  
  }  
  setClicks(newClicks)  
};
```

- Improvised stateHandlers

```
const handleLeftClick = () => setClicks({ ...clicks, left: clicks.left + 1 });  
vise versa for right
```

- Some readers might be wondering why we didn't just update the state directly, like this:?

```
const handleLeftClick = () => {  
  clicks.left++  
  setClicks(clicks)  
}
```

- Solution

- it is forbidden in React to mutate state directly, since it can result in unexpected side effects.

- Changing state has to always be done by setting the state to a new object.
- If properties from the previous state object are not changed, they need to simply be copied
- which is done by copying those properties into a new object and setting that as the new state.
-

- **Imp Note**

Storing all of the state in a single state object is a bad choice for this particular application;

Handling arrays

- Every click is stored in a separate piece of state called allClicks that is initialized as an empty array:

```
const [allClicks, setAll] = useState([]);
```

- Left - When the left button is clicked, we add the letter L to the allClicks array:

```
setAll(allClicks.concat('L'));
```

- Right - When the Right button is clicked, we add the letter R to the allClicks array:

```
setAll(allClicks.concat('R'));
```

Concat method is Immutable, it returns a brand New Array[] with the updated value; the **join method** on the allClicks array, that joins all the items into a single string, separated by the string passed as the function parameter, which in our case is an empty space.

- This is also one possibility but ,Why is this code fundamentally broken? (What rule does it break?)

```
const handleLeftClick = () => {  
  allClicks.push('L')  
  setAll(allClicks)  
  setLeft(left + 1)  
}
```

the state of React components, like allClicks, must not be mutated directly. Even if mutating state appears to work in some cases, it can lead to problems that are very hard to debug.

- React looks at the box (the memory reference), not what is inside the box.
- If you use .push(), it is the exact same box, so React ignores it.
- By using .concat(), you hand React a brand new box, and it instantly redraws the screen.
- Bonus Pro-Tip: The course teaches .concat(), which is perfectly correct. But in modern React jobs, you will actually see developers use the exact same Spread Operator trick you used for Objects:

```
setClicks([...clicks, 'L']);
```

- This means: "Create a new array, dump everything from the old clicks array into it, and then add 'L' at the end." Both methods do the exact same thing (create a new array), so use whichever one makes more sense to your brain!

Update of the state is asynchronous

- Course asked to create total state of left right clicked okay

```
const [clicks, setClicks] = useState({ left: 0, right: 0 })
const [total, setTotal] = useState(0);
> setTotal(clicks.left + clicks.right)
```

- Output

Left : 4 Right : 3

L L R L R R L

total : 6

- something wrong in output total should print 7 but it is printing 6;

Lets find out why in the upcoming sections Rendering clicks by value L:0 | R:0

left before 0 left after 0

Rendering clicks by value L:1 | R:0

- here clicked happend but value didnt changed in the actual clicks.left object
- but the in handleLeftClicked function its chnaging instantly

it should print after very first clicked: left before 0 left after 1

total should print 1 but it didnt.

but didnt chnage let see why ??

- The reason for this is that a state update in React happens **asynchronously**.
- not immediately but "at some point" after the current component function is finished, before the component is rendered again.
- By Mentor :-

```
setClicks({ ...clicks, left: clicks.left + 1 });
console.log('left after', clicks.left); // Prints the OLD number!
```

```
setTotal(clicks.left + clicks.right); // Uses the OLD number!
```

- In standard JavaScript, if you change a variable, it updates instantly on the very next line. React state does NOT work like that.
- When you run `setClicks(...)`, you are not instantly changing the `clicks` variable. You are just sending a request to the React Engine saying, "Hey, next time you redraw the screen, make the `clicks` variable equal this new number."
- But while the rest of your `handleLeftClick` function finishes running, the `clicks` variable is locked in time.
- It still holds the old data. That is why your `console.log` prints the old number, and your `setTotal` calculates using the old number!
- Because you declared it as a constant (`const [clicks, setClicks] = ...`), the value of `clicks` is completely locked in **scope for that specific run of the function**.

Solution

- You cannot rely on the state variable immediately after you "set" it
- Instead, you calculate the new number in a normal variable first, and then hand that normal variable to React.
- Final Verdict by Mentor on **Update of the state is asynchronous:-**
- So yes, you cannot rely on the **useState variable immediately after updating it**.
- You must "variablize" the update first, just like you did.

Conditional rendering

- It is just if/else logic!
- The `History` component renders completely different React elements depending on the state of the application. This is called conditional rendering.

Old React

- nothing special to note down just general reading material

Debugging React applications

- Good practices and tools for debugging are extremely important for this reason.
- Lucky for us, React is an extremely developer-friendly library when it comes to debugging.
- Before we move on, let us remind ourselves of one of the most important rules of web development.
 - The first rule of web development
 - **Keep the browser's developer console open at all times.**

- The Console tab in particular should always be open, unless there is a specific reason to view another tab. //
 - Keep both your code and the web page open together at the same time, all the time. //
 - we must transform our function into the less compact form and receive the entire props object without destructuring it immediately:
 1. Console.log is your Best Friend
 2. The Comma Rule (Never use +)
 - When you want to log an object, never do this: `console.log('The state is ' + clicks)`
 - Always use a comma: `console.log('The state is ', clicks)`
 3. The debugger; Command
 4. React Developer Tools
-

Rules of Hooks

1. The `useState` function must not be called from inside of a loop, a conditional expression, or any place that is not a function defining a component.
2. To recap, hooks may only be called from the inside of a function body that defines a React component:

```
const App = () => {
  // these are ok
  const [age, setAge] = useState(0)
  const [name, setName] = useState('Juha Tauriainen')

  if ( age > 10 ) {
    // this does not work!
    const [foobar, setFoobar] = useState(null)
  }

  for ( let i = 0; i < age; i++ ) {
    // also this is not good
    const [rightWay, setRightWay] = useState(false)
  }

  const notGood = () => {
    // and this is also illegal
    const [x, setX] = useState(-1000)
  }

  return (
    //...
  )
}
```

Event Handling Revisited

- Why is this illegal? Because you put parentheses () at the end. When React renders the HTML, it sees those parentheses and says, "Oh, you want me to execute this function right now!"
- It will run the function immediately when the page loads, **without waiting for the click!** If that function contains a set... state update, it will cause an infinite loop and crash your app.

1. THE GOOD WAY #1 (Pass by Reference)

```
<button onClick={handleLeftClick}>Click Me</button>
```

2. THE GOOD WAY #2: -> (Inline Arrow Function)

```
// CORRECT
<button onClick={() => console.log('clicked')}>Click Me</button>

// CORRECT
<button onClick={() => setClicks(clicks + 1)}>Click Me</button>
```

- This is also perfectly legal! You are not calling a function here; you are defining a brand new, nameless function right inside the HTML. React says, "Okay, I will save this nameless arrow function for later, and run it when the button is clicked."

A function that returns a function

Closures (Functions returning functions)

- It is a JavaScript concept called Currying or Closures.
- Another way to define an event handler is to use a function that returns a function.

```
const outer = () => {
  let count = 0;
  return function inner() {
    count++;
    console.log(count);
  };
}

const secrete = outer();
secrete();
```

- inner() function "remembers" the count variable from the outer scope, even after the outer function has finished running.
- React uses that exact same superpower to build customized event handlers.

- Instead of writing three different functions (setToOne, setToTwo, setToThree), you write ONE outer function that takes a parameter, and it returns a closure (an inner function) that remembers that parameter

```
const hello = () => {  
  const handler = () => console.log('hello world'); // This is a function  
  return handler; // It RETURNS the function!  
}
```

- When React sees

, it runs the hello() function immediately on render. But what does it get back? It gets the handler function back!

- So, after React executes hello(), the HTML basically transforms into this: `button`
- Because it successfully handed a function to the onClick, React is perfectly happy, and it will not cause an infinite loop.
- Syntaxes

1. Vanilla JS Syntax (Yours - Perfectly Valid)

```
const hello = () => {  
  return function handler() {  
    setMsg("Hello React")  
  }  
};
```

2. Standard React Arrow Syntax

```
const hello = () => {  
  const handler = () => {  
    setMsg("Hello React")  
  }  
  return handler;  
};
```

3. Ultra-Short Expert Syntax (Hard to read, but you will see it often)

```
const hello = () => () => setMsg("Hello React");
```

Passing Event Handlers to Child Components

```
const Button = ({ onClick, text }) => {
  return <button onClick={onClick}>{text}</button>
}

const App = () => {

  const [msg, setMsg] = useState("");

  const hello = () => () => setMsg("Hello React User , Welcome To react World ,
We are testing now A function that actually returns a function which basic Concept
in JavaScript Called 'Closures'");

  return (
    <div>
      // And used it like this:
      <Display textToShow={msg} />
      <Button onClick={hello()} text="Show Text" />
    </div>
  )
};
```

Do Not Define Components Within Components

```
// This is the right place to define a component
const Button = (props) => (
  <button onClick={props.onClick}>
    {props.text}
  </button>
)

const App = () => {
  const [value, setValue] = useState(10)

  const setToValue = newValue => {
    console.log('value now', newValue)
    setValue(newValue)
  }

  // Do not define components inside another component

  const Display = props => <div>{props.value}</div>

  return (
    <div>

      <Display value={value} />
      <Button onClick={() => setToValue(1000)} text="thousand" />
      <Button onClick={() => setToValue(0)} text="reset" />
    </div>
  )
};
```



```
    <Button onClick={() => setToValue(value + 1)} text="increment" />
  </div>
)
}
```

- Never define components inside of other components
 - The method provides no benefits and only leads to problems.
 - One such problem is that React will treat a component defined inside of another component as a "new component" in every render.
 - This makes it impossible for React to optimize the component.
-

Useful Reading

- The internet is full of React-related material. However, we use the new style of React for which a large majority of the material found online is outdated.
- The official React documentation is worth checking out at some point, although most of it will become relevant only later on in the course.
- - Also, everything related to class-based components is irrelevant to us;

Rendering a collection, modules ;

Rendering Collection

- Traditional Way :-

```
const App = ({notes}) => {  
  
  return (  
    <div>  
      <h1>Notes</h1>  
      <ul>  
        <li>{notes[0].content}</li>  
        <li>{notes[1].content}</li>  
        <li>{notes[2].content}</li>  
      </ul>  
    </div>  
  )  
};
```

- React Way :-

```
const App = ({ notes }) => {  
  
  return (  
    <div>  
      <h1>Notes</h1>  
      <ul>  
        {notes.map(note => <li>{note.content}</li>)}  
      </ul>  
    </div>  
  )  
};
```

Key-attribute

- Even though the application seems to be working, there is a nasty warning in the console:
- Each child in a list should have a unique "key" prop.
- in the error message suggests; the list items, i.e. the elements generated by the map method, must each have a unique key value: an attribute called key.

```
const App = ({ notes }) => {
```

```
return (  
  <div>  
    <h1>Notes</h1>  
    <ul>  
      {notes.map(note => <li key={note.id}>{note.content}</li>)}  
    </ul>  
  </div>  
)  
};
```

- And the error message disappears.
- React uses the **key attributes of objects in an array** to determine how to update the view generated by a component when the component is re-rendered.

Map

```
const result = notes.map(note => note.id)  
console.log(result)
```

- map always creates a new array, the elements of which have been created from the elements of the original array by mapping: using the function given as a parameter to the map method.

```
note => note.id
```

- Which is an arrow function written in compact form. The full form would be:

```
(note) => {  
  return note.id  
}
```

- The function gets a note object as a parameter and returns the value of its id field.

```
const result = notes.map(note => note.content)
```

- just we need used this snippet into react from

```
notes.map(note =>  
  <li key={note.id}>  
    {note.content}
```

```
    </li>
  )
```

- s used to create view elements, the value of the variable must be rendered inside curly braces.
- {} - curly braces is main factor of this section if you did not use wisely you will get in problem.

Anti-pattern: Array Indexes as Keys

- We could have made the error message on our console disappear by using the array indexes as keys.
- The indexes can be retrieved by passing a second parameter to the callback function of the map method:

```
const result = arr.map((ite, i) =>...);
```

- When called like this, `i` is assigned the value of the **index of the position** in the array where the note resides.

```
<ul>
  {notes.map((note, i) =>
    <li key={i}>
      {note.content}
    </li>
  )}
</ul>
```

- This is, however, **not recommended** and can create undesired problems even if it seems to be working just fine.

Refactoring Modules

- We'll separate displaying a single note into its own component Note:

```
const Note = ({ note }) => {
  return (
    <li>{note.content}</li>
  )
}

const App = ({ notes }) => {
  return (
    <div>
      <h1>Notes</h1>
      <ul>
```

```

    {notes.map(note =>
      <Note key={note.id} note={note} />
    )}
  </ul>
</div>
)
}

```

- The key attribute must ALWAYS be placed on the exact item that is being returned directly inside the `.map()` loop.
- Inside your component, your `.map()` loop is running like a factory conveyor belt. Every time the loop spins, it spits out a new box.

```

{notes.map(note =>
  <Note key={note.id} note={note} />
  // The factory stamps the ID on the OUTSIDE of the box here
)};

```

- React needs to be able to look at that conveyor belt and instantly identify every box. That is why the key MUST be stamped on the component itself.
- If you put the key on the
- inside the Note component; You are essentially hiding the ID card inside the box.

Refactoring Components

- A whole React application can be written in a single file. Although that is, of course, not very practical. Common practice is to declare each component in its own file as an ES6-module.
- In smaller applications, components are usually placed in a directory called components within the src directory. The convention is to name the file after the component. A possible directory structure for a project containing multiple components could be as follows:
- Do not forget to even we are separating Components into separate rule will be same for naming file convention first letter will always Capital like this `Note.jsx` , `Footer.jsx` ;

```

src/
├── main.jsx
├── App.jsx
└── components/           # Folder for reusable components
    ├── Footer.jsx       # File named after the component
    ├── Note.jsx
    └── Notification.jsx

```

- the main app logic and state stays trapped inside `const App = () => {}`. But each child component gets to hold its own tiny, specific logic too!
- `// Inside Note.jsx`

```
const Note = ({ note, onClickFn }) => {  
  
  // You can put tiny, specific logic just for the Note right here!  
  const capitalizedText = note.content.toUpperCase();  
  
  return (  
    <li onClick={onClickFn}>  
      {capitalizedText}  
    </li>  
  )  
}
```

Part2_a and Exercises 2.1 to 2.5 Veridict

- touches on the absolute core engine of React: The Virtual DOM.

Why is it mandatory?

- Imagine you have a list of 5 input boxes on the screen, and you type your name into the 3rd box. Then, you press a button that deletes the 1st box.
- Without keys, React gets confused. It looks at the new array and says: "Wait, there are only 4 boxes now. Let me just destroy all 5 boxes and redraw 4 new ones." If it does that, it deletes the name you just typed.
- With keys, React says: "Ah, Box ID #1 was deleted. But Box ID #2, #3, #4, and #5 are still here. I will just delete Box #1 and leave the others alone." Your text stays perfectly safe. Keys prevent massive UI bugs!

-
1. The Key Rule: The key goes on the absolute outermost box on the conveyor belt.
 2. The Curly Brace Rule: If you open an arrow function with `{}`, Return is God. If you forget it, you get the Invisible Man
- If you want to be a rebel and *skip the return keyword*, you have to use parentheses `()` instead of curly braces, which tells JavaScript to do an "implicit return". But honestly, just typing `return` is safer for now!

Forms

This is important since **we must never mutate state directly in React!**

The Save

- When the user clicks the `<button type="submit">`, the `<form onSubmit={addNote}>` triggers.
 - `event.preventDefault()`: If you don't have this, the browser will act like it's 1999 and try to refresh the whole webpage to send the data to a server. This line says "Stop! Don't reload the page. React will handle this."
 - **Creating the Object**: You create `newObj`. Notice how you set content: `newNote`. You don't have to go searching for what the user typed; you already have it sitting safely in your state from the Hijack!
 - **Saving to State**: You use `setNotes([...notes, newObj])` to safely add the new box to the conveyor belt without mutating the original array.
 - **Clearing the Input**: `setNewNote("")` instantly wipes the text box clean so the user can type another note.
-

The Hijack (Controlled Component)

- Look at your input box: In normal HTML, the input box controls itself. But here, you have hijacked it.
 - When the user types the letter "H" on their keyboard, the `onChange` event instantly fires.
 - It triggers `handleNoteChange(event)`.
 - `event.target.value` is JavaScript's way of reaching into the HTML input box and grabbing the letter "H".
 - It calls `setNewNote('H')`.
 - React instantly re-renders the screen, and forces the input box to display the value of the state (`value={newNote}`).
-

Filtering Displayed Elements

- Let's add some new functionality to our application that allows us to only view the important notes.
 - just showed how to use Array Method called `filter`.
-

Exercise Veridict (Summury)

Early Return "trick" to kill a function;

- (e.g)

```
const addPerson = (event) => {
  event.preventDefault();

  let isDuplicateName = persons.some((person) => person.name === newName);

  if (isDuplicateName) {
    alert(`${newName} is already added to phonebook`);
    return
  };

  const personObject = {
    name: newName
  }

  setPersons([...persons, personObject]);
  setNewName('')
};
```

- You didn't lack JavaScript concepts. You just hadn't seen the Early Return "trick" used to stop a function before. You knew what return does (you use it in every React component). You just didn't realize you were allowed to use a blank return; as a kill switch.
- if I did not put return after `alert` it will continuously goes down read remaining script; unfortunately app will get crash;
- Practice it into vanilla JS :-

```
filter(),
map()
reduce(),
some()
```


Getting data from server

- For a while now we have only been working on **"frontend", i.e. client-side (browser) functionality**. We will begin working on **"backend", i.e. server-side functionality** in the *third part of this course*. Nonetheless, we will now take a step in that direction by familiarizing ourselves with how the code executing in the browser communicates with the backend.
- we are going to use a tool meant to be used during software development called **JSON Server** to act as our **fake server**.
- so first create **db.json** file into root folder which I am working in **part2_c => getting-data-from-server** so create here all folders and files are present like **vite-setup,src**, so create here **db.json** here.

```
{
  "notes": [
    {
      "id": "1",
      "content": "HTML is easy",
      "important": true
    },
    {
      "id": "2",
      "content": "Browser can execute only JavaScript",
      "important": false
    },
    {
      "id": "3",
      "content": "GET and POST are the most important methods of HTTP protocol",
      "important": true
    }
  ]
}
```

- put this data into **db.json**.
- And start the fake server with help of tool called **JSON-Server**.
- **npx json-server --port 3001 db.json**.
- redirect into browser **localhost:3001/notes**.
- Extra info :-
 - (A "port" is just a numbered door on your computer. Your computer has exactly 65,535 doors.)
 - You should never use the port that your Vite frontend is running on (usually 5173 or 3000). If your frontend and backend try to share the exact same port, your computer will crash.
- Going forward, the idea will be to save the notes to the server, which in this case means saving them to the json-server. The **React code fetches** the notes from the **server** and **renders** them to the

screen. Whenever a new note is added to the application, the React code also sends it to the server to make the new note persist in "memory".

The browser as a runtime environment

The Problem: JavaScript only has one brain.(single-thread)

- JavaScript is **"single-threaded"**. Imagine JavaScript is a chef in a kitchen, but the chef only has one hand (single-thread). They can only do exactly one task at a time. If the chef puts a pizza in the oven and just stands there staring at it for 20 minutes waiting for it to finish, the entire restaurant shuts down. No other food gets cooked.
- If your React app asks the json-server for data, and it takes 3 seconds to download... if JavaScript just stood there waiting, your entire website would freeze for 3 seconds. You couldn't scroll, you couldn't click buttons, it would look like it crashed.

The Solution: Asynchronous Callbacks

- Because we don't want the website to freeze, JavaScript uses Asynchronous Code. Instead of staring at the oven, the chef puts the pizza in the oven, sets a timer, and goes back to chopping vegetables. When the timer rings (a "callback"), the chef knows the pizza is done and takes it out.
 - What this means for your code:
 - When you tell React to connect to your http://localhost:3001/notes server, you don't write it like normal code. You use a "Promise" (a timer).
 - You basically tell React: "Hey, go start downloading this data. But don't wait for it! Keep drawing the website. .then() when the data finally arrives, run this function to update the screen."
-

npm

1. What is NPM?

- NPM stands for Node Package Manager. Think of NPM as the App Store for developers.

You don't have to build every single tool from scratch. Thousands of smart developers have already written tools for you, and put them on NPM for free. When you run `npm install axios` in your terminal, it reaches out to the "App Store", downloads the code for Axios, and puts it in your massive `node_modules` folder. It also writes the name of it inside your `package.json` file, so your app remembers that you installed it!

- `npx` stands for **Node Package Execute**.
 - `npm` is for installing things permanently on your hard drive.
 - `npx` is for running things.
- **The Magic of `npx`**

- Normally, to run a program, you have to download it, install it, and keep it on your computer forever.
- **npx has a superpower:** if you ask it to run a tool that you do not own, it will secretly reach up to the internet, download a temporary copy of the tool, run it one time, and then delete it so it doesn't clutter your hard drive!

2. Why Axios instead of Fetch?

- `fetch()` is a tool that is built directly into every web browser. It works, but it is annoying and requires extra chores.
- When a database sends data across the internet, it sends it as a massive, ugly string of text (raw JSON). If you use `fetch()`, you have to write an extra line of code to manually translate that raw text back into a JavaScript array:

```
// The Fetch Way (Annoying)
fetch('http://localhost:3001/notes')
  .then(response => response.json()) // 🚨 Extra chore required here!
  .then(data => console.log(data))
```

- Axios is a tool you download from the NPM App Store that does the chores for you. Axios automatically translates the raw text into a beautiful JavaScript array behind the scenes. **It also handles errors (like 404 Not Found) much better than fetch does.**

```
// The Axios Way (Clean)
axios.get('http://localhost:3001/notes')
  .then(response => console.log(response.data)) // Automatically translated!
```

- Another good reason to use Axios is that it helps us get familiar with adding external libraries, or npm packages, to React projects.
- **npm install axios**

you will find this into dependencies - the dependencies part is of most interest to us as it defines what dependencies, or external libraries, the project has.

```
"dependencies": {
  **"axios": "^1.19.0",**
  "react": "^19.2.8",
  "react-dom": "^19.2.8"
},
```

- **npm install json-server --save-dev**

Install json-server as a development dependency (only used during development)

- and making a small addition to the scripts part of the package.json file:

```
{
  // ...
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview",
    **"server": "json-server -p 3001 db.json" >>> add this line into scripts**
  },
}
```

- We can now conveniently, without parameter definitions, start the json-server from the project root directory with the command:

npm run server this will generate Endpoints: <http://localhost:3001/notes>

Axios and promises

- how to get data using axios

```
import axios from 'axios'
const promise = axios.get('http://localhost:3001/notes')
```

- Axios' method get returns a promise
 - In other words, a promise is an object that represents an asynchronous operation. A promise can have three distinct states:
 - **The promise is pending:** It means that the asynchronous operation corresponding to the promise has not yet finished and the final value is not available yet.
 - **The promise is fulfilled:** It means that the operation has been completed and the final value is available, which generally is a successful operation.
 - **The promise is rejected:** It means that an error prevented the final value from being determined, which generally represents a failed operation.
- If we want to access the result of the operation represented by the promise, we must register an event handler to the promise. This is achieved using the method then:

```
const promise = axios.get('http://localhost:3001/notes')

promise.then(response => {
  console.log(response)
})
```

- Improvised version but still problematic

```
axios
  .get('http://localhost:3001/notes')
  .then(response => {
    const { data } = response;
    createRoot(document.getElementById('root')).render(
      <StrictMode>
        <App notes={data} />
      </StrictMode>,
    )
  })
```

- The course says: "What's not immediately obvious is where axios.get should be placed."
 - If you just copy-paste axios.get() directly into the middle of your App.jsx file, it will cause a catastrophic Infinite Loop. Every time you type a letter in an input box, React re-renders the page... which would trigger axios.get() again... which updates the state... which triggers a re-render... which triggers axios.get() again. Your computer will crash.

Effect-hooks (useEffect)

- As per the official docs:
 - Effects let a component connect to and synchronize with external systems. This includes dealing with network, browser DOM, animations, widgets written using a different UI library, and other non-React code.
 - As such, effect hooks are precisely the right tool to use when fetching data from a server.
- so we do need to pass data as prop to App in main.jsx remove all stuff except rednering block.
- Initial code block of hook (Syntax)

```
import { useState, useEffect } from "react";
import axios from "axios";

const App = () => {

  const [notes, setNotes] = useState([]);

  useEffect(() => {
    console.log('useEffect');
    axios
      .get('http://localhost:3001/notes')
      .then(response => {
        console.log('promise fulfilled');
        setNotes(response.data);
      })
  })
```

```
    }, []);

    console.log('render', notes.length, 'notes')
  };

  export default App;
```

- Output

```
render 0 notes
useEffect
promise fulfilled
render 3 notes
```

- First, the body of the function defining the component is executed and the component is rendered for the first time. At this point render 0 notes is printed, meaning data hasn't been fetched from the server yet.
 - The following function, or effect in React parlance:

```
() => {
  console.log('effect')
  axios
    .get('http://localhost:3001/notes')
    .then(response => {
      console.log('promise fulfilled')
      setNotes(response.data)
    })
}
```

- is executed immediately after rendering. The execution of the function results in effect being printed to the console, and the command *axios.get* initiates the fetching of data from the server **as well as registers the following function as an event handler for the operation:**
- When data arrives from the server, the JavaScript runtime calls the function registered as the event handler, which prints promise fulfilled to the console and stores the notes received from the server into the state using the function `setNotes(response.data)`;
- As always, a call to a state-updating function triggers the re-rendering of the component. As a result, render 3 notes is printed to the console, and the notes fetched from the server are rendered to the screen.
- Finally, let's take a look at the definition of the effect hook as a whole:

```
useEffect(() => {
  console.log('effect')
```

```

    axios
      .get('http://localhost:3001/notes').then(response => {
        console.log('promise fulfilled')
        setNotes(response.data)
      })
  }, [])

```

- Let's rewrite the code a bit differently.

```

const hook = () => {
  console.log('effect')
  axios
    .get('http://localhost:3001/notes')
    .then(response => {
      console.log('promise fulfilled')
      setNotes(response.data)
    })
}
useEffect(hook, []);
`**** useEffect(hook, []) ****`

```

- Now we can see more clearly that the function `useEffect` takes two parameters. The first is a function, the effect itself.
 - By default, effects run after every completed render, but you can choose to fire it only when certain values have changed.
- So by default, the effect is always run after the component has been rendered. In our case, however, we only want to execute the effect along with the first render.
- The second parameter of `useEffect` is used to specify how often the effect is run. If the second parameter is an empty array `[]`, then the effect is only run along with the first render of the component.
- Note that we could have also written the code for the effect function this way:

```

useEffect(() => {
  console.log('effect')

  const eventHandler = response => {
    console.log('promise fulfilled')
    setNotes(response.data)
  }

  const promise = axios.get('http://localhost:3001/notes')
  promise.then(eventHandler)
}, [])

```

- That is very clunky. It takes 3 steps and creates useless variables.

- Always use this shorter version unless major requirement

```
useEffect(() => {  
  console.log('effect')  
  axios  
    .get('http://localhost:3001/notes').then(response => {  
      console.log('promise fulfilled')  
      setNotes(response.data)  
    })  
}, [])
```

All About hook `useEffect(function,[])`;

- **Parameter 1: The Function (The "What")**

- The first parameter is just the code you want React to run.

Example: "Go fetch the data from the backend using Axios."

- **Parameter 2: The Array (The "When")**

- This is called the **Dependency Array**, and it is the most important part of `useEffect`. It tells React exactly **when it is allowed to run the function**. You have three choices:

- **Choice A: An Empty Array**

The **Empty Array []**, you pass an empty array, you are telling React: "Run this function exactly **ONE TIME** when the **website first loads**. Never run it again. This is perfect for fetching your initial database. You only need to download the notes once when the user opens the page!

- **Choice B: An Array with a variable inside [searchQuery]**

If you put a state variable inside the array, you are telling React: "Run this function when the page loads, **AND run it again every single time the searchQuery state changes**." (You will use this later in the course).

- **Choice C: No Array at all (DANGER 🚨)**

If you forget to type the array completely, you are telling React: "**Run this function every single time the component re-renders**." Because **fetching data updates the state, and updating the state causes a re-render**, this will cause a catastrophic **Infinite Loop** that will crash your browser.

The development runtime environment

1. The JavaScript code making up our React application is run in the browser.
2. The browser gets the JavaScript from the **React dev server**, which is the application that runs after running the command **npm run dev**
3. The dev-server transforms the JavaScript into a format understood by the browser.

4. Among other things, it stitches together JavaScript from different files into one file.
- The React application running in the browser fetches the JSON formatted data from json-server running on port 3001 on the machine. The server we query the data from - json-server - gets its data from the file db.json.

Final Verdict of this chapter (Course + Exercises)

- npm installs it permanently in your project, npx borrows it temporarily or executes it instantly.
- First always check for dependencies like axios or json-server.
- **npm install axios**
- **npm install json-server --save-dev** (directly get saved in dev dependencies) / for temporary **npx json-server --port 3001 db.json**
- **"server": "json-server -p 3001 db.json"** <= add this line into script section of the package.json
- **npm run server**
- react has habit or nature of re-rendering, if got indication (flare gun) from **useState** so it starts re-rendering all thing so this axios.get() method will get stuck into permanent loop so use empty array [] as a one time subscription for hook Effect.
- If we put **axios.get() fetch request** inside main.jsx (and put the ReactDOM.render() code inside the .then() block), we force React to sit there in the dark with its hands tied behind its back, **waiting for the API to finish its 5-second download**. The user sees nothing but a white screen.
- By putting the fetch inside with **useEffect**, React is allowed to draw rest of the components. Then, 5 seconds later, the data pops into the list.

Altering Data In Server

- **React** -> **talks to** -> **json-server** -> **edits** -> **db.json**
- *axios* is middle man between **React** and **Server**;

Important note on hook useState

- why useState initialized with empty array [] or empty string '' ? what will happen if we init with blank state
- Millisecond 1: React instantly draws the screen.
- Millisecond 2: useEffect triggers the database download.
- Look at the bottom of your code. At Millisecond 1, React is forced to read this line of code:
notesToShow.map(note => ...);

If you left your state completely blank (useState()), **then notes would be undefined**. When JavaScript tries to run .map() on undefined, it completely panics. **It throws a massive red error: TypeError: Cannot read properties of undefined (reading 'map')** and your entire website crashes before the database even has a chance to respond!

By initializing it with an empty array [], you are giving React a safe placeholder. At Millisecond 1, React runs .map() on the empty array. **It says, "Okay, there are 0 things in this array, so I will draw 0 things on the screen."** It doesn't crash! It just waits peacefully until the data arrives a second later.

REST

- In REST terminology, we refer to individual data objects, such as the notes in our application, as resources.
- Every resource has a unique address associated with it - its URL.
- According to a general **convention used by json-server**, we would be able to locate an individual note at the resource URL **notes/3**, where **3 is the id** of the resource.
 - Resources are fetched from the server with HTTP GET requests.
 - For instance, an HTTP GET request to the URL notes/3 will return the note that has the id number 3.
 - An HTTP GET request to the notes URL would return a list of all notes.
- Creating a new resource for storing a note is done by making an HTTP POST request to the notes URL according to the REST convention that the json-server adheres to.
 - The data for the new note resource is sent in the body of the request
 - json-server requires all data to be sent in JSON format

- What this means in practice is that the data must be a correctly formatted string and that the request must contain the *Content-Type request header* with the *value application/json*.

Sending Data to the Server

- In previous part we successfully understand how to fetch data by using **React-hook Effect** or **useEffect** with `axios.get()` method.

```
useEffect(() => {
  axios
    .get('http://localhost:3001/notes')
    .then(response => {
      const { data } = response;
      setNotes(data)
    })
}, []);
```

- Now it's time to send the data to the server, we have already *fake json-server* with *db.json*, `axios.post()` method will help us to send the data to that server okay.

```
const addNote = (event) => {
  event.preventDefault();

  const newObject = {
    content: newNote,
    important: Math.random() < 0.5
  };

  // Adding Note to the server (fake-db) and to the local-machine via
  axios.POST() Method
  axios
    .post('http://localhost:3001/notes', newObject) // its take to param
    one is URL and second is response
    .then(response => {
      const { data } = response;
      setNotes([...notes, data])
      setNewNote('')
      console.log(response)
    })
};
```

Changing the Importance of Notes (manipulating data by **axios.put()**)

- `axios.put(location, changedData)`

```
const toggleImportanceOf = id => {
  const url = `http://localhost:3001/notes/${id}`
  const note = notes.find(n => n.id === id)
  const changedNote = { ...note, important: !note.important }

  axios
    .put(url, changedNote)
    .then(response => {
      setNotes(notes.map(note => note.id === id ? response.data : note))
      // response.data is changedNote here (new)
    })
}
```

- the first line defines the unique URL for each note resource based on its id.
- The array find method is used to find the note we want to modify, and we then assign it to the note variable.
- After this, we create a new object that is an exact copy of the old note, apart from the important property that has the value flipped (from true to false or from false to true)
- the `important` property gets the negation of its previous value in the original object
- After that its time to change in db so, we are invoking `axios.put()` method here to make change in `db.json`. `put(location, newData)` took two arguments:

```
setNotes(notes.map(note => note.id === id ? response.data : note))
```

- so here we are manipulating data into `main state` by map method by matching its id, `response.data` is acting behind the scenes is `newData`: `note` means old data;
- The map method creates a new array by mapping every item from the old array into an item in the new array. In our example, the new array is created conditionally so that **if `note.id === id` is true**; the note object returned by the server is added to the array. **If the condition is false, then we simply copy the item from the old array into the new array instead.**

Extracting Communication with the Backend into a Separate Module

- Right now, your `App.jsx` file is doing too many jobs. It is drawing HTML, it is managing React State, it is filtering arrays, and it is talking to the backend database using Axios. In the real world, this makes `App.jsx` massive and impossible to read.
- This section teaches you a massive architectural upgrade: **Separation of Concerns**.
- This file is what we call a Service (which is why we put it in a `services` folder). A Service is just a helper file that does the dirty work of talking to a database.

- The App component has become somewhat bloated after adding the code for communicating with the backend server. In the spirit of the single responsibility principle, we deem it wise to extract this communication into its own module.

Cleaner Syntax for Defining Object Literals

- old way of exporting

```
export default {  
  getAll: getAll,  
  create: create,  
  update: update  
}
```

- new way of exporting (ES6+ feature)

```
export default {getAll,create,update}
```

Promises and Errors

- We had previously mentioned that a promise can be in one of three different states. When an axios HTTP request fails, the associated promise is rejected. Our current code does not handle this rejection in any way.
- The rejection of a promise is handled by providing the then method with a second callback function, which is called in the situation where the promise is rejected.
- The more common way of adding a handler for rejected promises is to use the catch method.
- In practice, the error handler for rejected promises is defined like this:

```
axios  
  .get('http://example.com/probably_will_fail')  
  .then(response => {  
    console.log('success!')  
  })  
  .catch(error => {  
    console.log('fail')  
  })
```

- If the request fails, the event handler registered with the catch method gets called.

Exercises

- mess

```
import { useState, useEffect } from "react";
import personServices from './services/persons';

const Filter = ({ value, handler }) => {
  return (
    <>
      Filter shown with <input value={value} onChange={handler} />
    </>
  )
};

const PersonForm = ({ onSubmit, newName, userNameHandler, newNumber,
  userNumberHandler }) => {
  return (
    <form onSubmit={onSubmit} >

      <div>
        name: <input value={newName} onChange={userNameHandler} />
      </div>

      <div>
        number : <input value={newNumber} onChange={userNumberHandler} />
      </div>

      <div>
        <button type="submit">add</button>
      </div>

    </form>
  )
};

const Persons = ({ personsToShow, deleteHandler }) => {
  return (
    <div>
      {personsToShow.map(person => {
        const { id, name, number } = person;
        return (
          <p key={id}>{name} {number} <button onClick={() => deleteHandler(id,
name)}>delete</button></p>
        )
      })}
    </div>
  )
};

const App = () => {

  //
  const [persons, setPersons] = useState([]);
  // Locked html Inputs into React state here;
```

```

const [newName, setNewName] = useState('');
const [newNumber, setNewNumber] = useState('');
const [searchQuery, setSearchQuery] = useState('');

// fetching data here
useEffect(() => {
  personServices
    .getAll()
    .then(data => {
      setPersons(data)
    })
}, [])

const addPerson = (event) => {

  event.preventDefault();

  let isDuplicateName = persons.find((person) => person.name === newName);

  // Duplicate Checking
  if (isDuplicateName) {

    if (isDuplicateName.number !== newNumber) {

      let confirmation = window.confirm(`${newName} is already added to
      phonebook, replace the old number with a new one ?`);

      if (confirmation) {
        const { id } = isDuplicateName;
        const changedNumber = { ...isDuplicateName, number: newNumber };
        personServices
          .update(id, changedNumber)
          .then(updatedPerson => {
            setPersons(persons.map(person => person.id === id ? updatedPerson :
            person))
            setNewName('')
            setNewNumber('')
          })
      } else {
        setNewName('')
        setNewNumber('')
      }
    } else {
      alert(`${newName} is already added to phonebook`);
      setNewName('');
      setNewNumber('')
    }
    return
  };

  // object that contains user input
  const personObject = {
    name: newName,
    number: newNumber
  }

```

```

    };

    // Connection to the backend (fake-server)
    personServices
      .create(personObject)
      .then(returnedPerson => {
        setPersons([...persons, returnedPerson])
        setNewName('')
        setNewNumber('')
      })
    };

    //
    const deletePersonOf = (id, name) => {
      if (window.confirm(`Delete ${name}`)) {
        personServices
          .remove(id)
          .then(() => (setPersons(persons.filter(person => person.id !== id))))
      }
    };

    // handler for name
    const handleUserName = (event) => {
      setNewName(event.target.value)
    };

    // handler for number
    const handleUserNumber = (event) => {
      setNewNumber(event.target.value)
    };

    // handler for searchBar
    const handleUserQuery = (event) => {
      setSearchQuery(event.target.value)
    };

    // Filtration on typed
    const personsToShow = persons.filter(person =>
    person.name.toLowerCase().includes(searchQuery.toLowerCase()));

    return (
      <div>
        <h1>Phonebook</h1>

        <Filter value={searchQuery} handler={handleUserQuery} />

        <h2>add a new</h2>

        <PersonForm onSubmit={addPerson} newName={newName} userNameHandler=
        {handleUserName} newNumber={newNumber} userNumberHandler={handleUserNumber} />

        <h2>Numbers</h2>
        <Persons personsToShow={personsToShow} deleteHandler={deletePersonOf} />
      </div>
    );
  }
}

```



```

    </div>
  )
};

export default App;

```

there is no replacement for if block but by returning we can skip else if condition is not so hard

- improvised

```

import { useState, useEffect } from "react";
import personServices from './services/persons';

const Filter = ({ value, handler }) => {
  return (
    <>
      Filter shown with <input value={value} onChange={handler} />
    </>
  )
};

const PersonForm = ({ onSubmit, newName, userNameHandler, newNumber,
  userNumberHandler }) => {
  return (
    <form onSubmit={onSubmit} >

      <div>
        name: <input value={newName} onChange={userNameHandler} />
      </div>

      <div>
        number : <input value={newNumber} onChange={userNumberHandler} />
      </div>

      <div>
        <button type="submit">add</button>
      </div>

    </form>
  )
};

const Persons = ({ personsToShow, deleteHandler }) => {
  return (
    <div>
      {personsToShow.map(person => {
        const { id, name, number } = person;
        return (
          <p key={id}>{name} {number} <button onClick={() => deleteHandler(id,
            name)}>delete</button></p>

```

```

    )
  })}
</div>
)
};

const App = () => {

  // ==== useState ==== //
  const [persons, setPersons] = useState([]);
  // name
  const [newName, setNewName] = useState('');
  // number
  const [newNumber, setNewNumber] = useState('');
  // searchInput
  const [searchQuery, setSearchQuery] = useState('');

  // ==== fetching data here ==== //
  useEffect(() => {
    personServices
      .getAll()
      .then(data => {
        setPersons(data)
      })
  }, [])

  // ==== Functions ==== //
  const addPerson = (event) => {
    event.preventDefault();

    // Finding Entry here
    let isDuplicate = persons.find((person) => person.name === newName);

    // Checking for duplicate
    if (isDuplicate && isDuplicate.number === newNumber) {
      alert(`${newName} is already added to phonebook`);
      setNewName('');
      setNewNumber('');
      return;
    };

    // Cheking for non duplicate
    if (isDuplicate && isDuplicate.number !== newNumber) {
      let confirmation = window.confirm(`${newName} is already added to phonebook, replace the old number with a new one ?`);

      if (confirmation) {
        let changedNumber = { ...isDuplicate, number: newNumber };
        personServices
          .update(isDuplicate.id, changedNumber)
          .then(updatedPerson => {
            setPersons(persons.map(p => p.id === isDuplicate.id ? updatedPerson :
p))
            setNewName('')

```

```

        setNewNumber('')
      })
    }
    else {
      setNewName('')
      setNewNumber('')
    }
    return
  }

  // object that contains user input
  const personObject = {
    name: newName,
    number: newNumber
  };

  // Connection to the backend (fake-server)
  personServices
    .create(personObject)
    .then(returnedPerson => {
      setPersons([...persons, returnedPerson])
      setNewName('')
      setNewNumber('')
    })
  };

  //
  const deletePersonOf = (id, name) => {
    if (window.confirm(`Delete ${name}`)) {
      personServices
        .remove(id)
        .then(() => (setPersons(persons.filter(person => person.id !== id))))
    }
  };

  // Filtration on typed
  const personsToShow = persons.filter(person =>
person.name.toLowerCase().includes(searchQuery.toLowerCase()));

  // ==== Handlers ==== //
  // handler for name
  const handleUserName = (event) => {
    setNewName(event.target.value)
  };

  // handler for number
  const handleUserNumber = (event) => {
    setNewNumber(event.target.value)
  };

  // handler for serachBar
  const handleUserQuery = (event) => {
    setSearchQuery(event.target.value)
  };

```

```

};

return (
  <div>
    <h1>Phonebook</h1>

    <Filter value={searchQuery} handler={handleUserQuery} />

    <h2>add a new</h2>

    <PersonForm onSubmit={addPerson} newName={newName} userNameHandler=
{handleUserName} newNumber={newNumber} userNumberHandler={handleUserNumber} />

    <h2>Numbers</h2>
    <Persons personsToShow={personsToShow} deleteHandler={deletePersonOf} />
  </div>
)
};

export default App;

```

Mentor session and Verdict of this chapter

- Question 1: Explain the difference between POST, PUT, and DELETE in REST:
 - Which ones need a data object (body) sent with them, and which one only needs a URL with an id? What is the specific job of each method on the backend?

q.1 =>

- post and put both needed body , post save new entry from newly typed userinput , put just needed updated body with id (location) , delete needs id only
 - newly we used hardcoded data okay but in real case cant save this hardcoded data for while beacuse we need modify them and reder to internet back , so should be saved somewhere on internet okay , course introduced me backend fake-server okay so puted that data into that to acces that we have method get , post , put , delete there are four main operations okay get we getting fetching data from internet where we saved post we are taking data from browser react app and then send to server put updating data on buisness logic delete we destroying that entry from db or internet permmently
- Mentor hitted me with this
 1. Question 1: HTTP Methods (100% - PERFECT)
 2. Your answer here was rock-solid:
 - **POST:** Sends a new body to create an entry.
 - **PUT:** Sends an updated body + targeting ID to replace/update an entry.

- **DELETE:** Needs only the target ID URL to destroy an entry.
- **GET:** Reads/fetches data.
- You completely understand REST CRUD. Flawless.

- Question 2: React Immutability & Array Methods

- When syncing our React state after a server update:
 - Why do we use `.map()` with a ternary operator (? 😊 when updating an item (PUT)?
 - Why do we use `.filter()` with `!==` when removing an item (DELETE)?
 - Why are we forbidden from just doing `persons.splice()` or `persons[i] = newPerson?`

q.2 =>

- well my basics are clear about arrays but this question seems to me new why we do use this methods
- well as i know its new es6+ feature that handle traditional chores of mapping data and replace if else ladder ,
- well again this are all high ordered functions automate lots of things i mean save lot line of code we can use vanilla method to filter out things
- same answer i mean if js giving us power of mutating data very easy way why we would take long way ??
- Mentor hit me with this

1. Question 2: Immutability & Array Methods (MISSED - CRITICAL REACT CONCEPT)

2. You answered that `.map()` and `.filter()` are just shorter ES6 syntax. While they are cleaner, *that is NOT the reason* **React forces us to use them.**

3. Here is the golden rule of **React: NEVER MUTATE STATE DIRECTLY.**

4. Why can't we use `persons.splice()` or `persons[0] = newObj?`

Methods like `.splice()`, `.push()`, or direct assignment `persons[0] = ...` mutate (modify) the existing array in-place in memory.

React checks if state changed by looking **at the memory reference of the array.** *If you modify the same array in place, React thinks: "The memory address is the same, nothing changed!" and it will NOT re-render the screen.*

5. Why do we use `.map()` and `.filter()`?

Both `.map()` and `.filter()` **never touch the old array. They create and return a BRAND NEW array in memory.** new Array means new memory reference and this reference react gets from `useState` , hook state is `useState` for react and react re-renders the screen instantly!

6. **Rule to remember:** In React, `.map()` and `.filter()` are used because **they are pure / immutable** — they *return fresh copies instead of mutating existing state*.

- Question 3: Services & Separation of Concerns

Why do we extract our network requests into a separate file (`services/persons.js`) and return `response.data` instead of writing raw `axios.get()` or `axios.post()` directly inside `App.jsx`? What problem does this solve?

q.3 =>

- mostly App component will get massive time by time and We are if overrulling DRY so just defined once and accese whener we wanted and it will prevents from clutter may be noy sure correct me here if wrong

- Mentor hitted me with this

0. Question 3: Services & Separation of Concerns (85% - VERY GOOD)

1. You nailed the main points: preventing `App.jsx` from becoming massive, reducing clutter, and following DRY (Don't Repeat Yourself).

2. The extra bonus point: **It gives us Separation of Concerns.**

- **App.jsx should only care about UI & User Interaction** (*buttons, forms, state*).
 - **services/persons.js should only care about HTTP & Network Protocols** (*Axios, URLs, formatting response.data*)
 - If your backend URL changes tomorrow from `localhost:3001` to `api.myphonebook.com`, you only have to edit one line in `persons.js`, and `App.jsx` never has to change at all.
-

- Question 4: Promises and Errors (`.catch`)

Imagine you try to update a person's phone number using `axios.put()`, but someone else deleted that person from the database 5 seconds ago (the server responds with a 404 Not Found error):

- What happens to your `.then()` block?
- What block runs instead, and what two things should you do inside of it to clean up the UI?

q.4 =>

- not sure
- handle the erroe by `.error ()` blockGame belowe above
- Mentor hitted me with this

1. Question 4: Promises and Errors (MISSED - NEEDS REINFORCEMENT)

2. Let's review what happens when an API call fails (like a 404 Not Found):

3. What happens to `.then()`?

* It is completely skipped. **JavaScript will not run a single line inside `.then()`**.

2. What runs instead?

* The `.catch(error => { ... })` block (remember the method name is `.catch()`, not `.error()`).

3. What 2 things should you do inside `.catch()`?

* **Notify the user:** `alert("This person was already removed from the server")`

* **Clean the UI:** **Remove the ghost person from React state** so the UI matches reality: `setPersons(persons.filter(p => p.id !== id))` why ? `*db changes*` nothing to do with `*reactState*` we have to take care of it

Adding styles to React app

class vs className

- **JSX is NOT real HTML.** *JSX is just fancy JavaScript disguised to look like HTML.* When Vite builds your project, it *converts all your JSX* into pure, **raw JavaScript functions**.
- **The Problem with class in JavaScript :**
 - In JavaScript, **class is a reserved keyword** used for Object-Oriented Programming (like creating ES6 classes):

```
class User {  
  constructor(name) {  
    this.name = name;  
  }  
}
```

- Because class is already a strictly reserved word in JavaScript, the creators of React could not use class as an attribute in JSX without causing a massive conflict in the JavaScript engine!
- React converts className="note" back into real HTML:
- !

Improved error message

- Learnt Here how to replace ugly browser alerts into Beautiful notification with logic.

```
const Notification = ({ message }) => {  
  if (message === null) {  
    return null  
  }  
  
  return (  
    <div className="error">  
      {message}  
    </div>  
  )  
}  
export default Notification
```

- In React, **if a component returns null, React completely removes it from the browser DOM.** Zero HTML is drawn!

Inline Styles

- CSS rules are defined slightly differently in JavaScript than in normal CSS files
- traditional Way (**regular CSS, is that hyphenated (kebab case)**)

```
{  
  color: green;  
  font-style: italic;  
}
```

- React Way (object) - (**CSS properties are written in camelCase.**)

```
{  
  color: 'green',  
  fontStyle: 'italic'  
}
```

E.g Styles Component in react way

```
const Footer = () => {  
  const footerStyle = {  
    color: 'green',  
    fontStyle: 'italic'  
  }  
  
  return (  
    <div style={footerStyle}>  
      <br />  
      <p>  
        Note app, Department of Computer Science, University of Helsinki 2025  
      </p>  
    </div>  
  )  
}  
  
export default Footer
```

- Inline styles comes with certain limitations.
 1. **No Pseudo-classes (:hover, :active, :focus):** You cannot write hover effects with inline styles! In CSS, you write `button:hover { background: red }`. In an inline style object, `:hover` does not exist.
 2. **No Media Queries (Responsive Mobile Design)** 🤔 * You cannot write `@media (max-width: 768px)` in an inline style object to change styles on mobile phones vs desktop screens.

3. **No Keyframe Animations:** You cannot define CSS @keyframes animations directly inside a `style={{} }` object.
4. **Code Clutter & Performance:** *If a component needs 20 styles, your JSX becomes massive and unreadable, and JavaScript has to create a brand new style object in memory on every single render.*
5. **Takeaway:** Use inline styles only for tiny, dynamic things (like changing a progress bar width `style={{ width:${percent}% }}`). For real design, use external CSS files, CSS Modules, or Tailwind.

Index.css vs App.css

- index.css (Global Styles for the entire website)
 - Where it is imported: Inside main.jsx (import './index.css').
 - What goes inside it: Global, baseline styles that apply to the entire webpage, such as:
 - Setting the font family for the whole website: `body { font-family: sans-serif; }`
 - CSS resets / box-sizing: `* { box-sizing: border-box; margin: 0; }`
 - Global CSS variables (`:root { ... }`)
 - Global utility classes (like your `.error` or `.success` notification classes)
 - App.css (Styles specifically for the App Component)
 - Where it is imported: Inside App.jsx (import './App.css').
 - What goes inside it: Styles that specifically belong to the layout of , such as:
 - Centering the main app container: `.app-container { max-width: 600px; margin: auto; }`
 - Vite's demo animations / logos
-